

TouchDesigner上級編 1 - TouchDesigner + Shader (GLSL) 入門

今日の内容

- TouchDesignerでShader (GLSL) を試してみる
- Shaderとは?
- TouchDesignerでShaderを表示
- フラグメントシェーダー (fragment shader) の基本

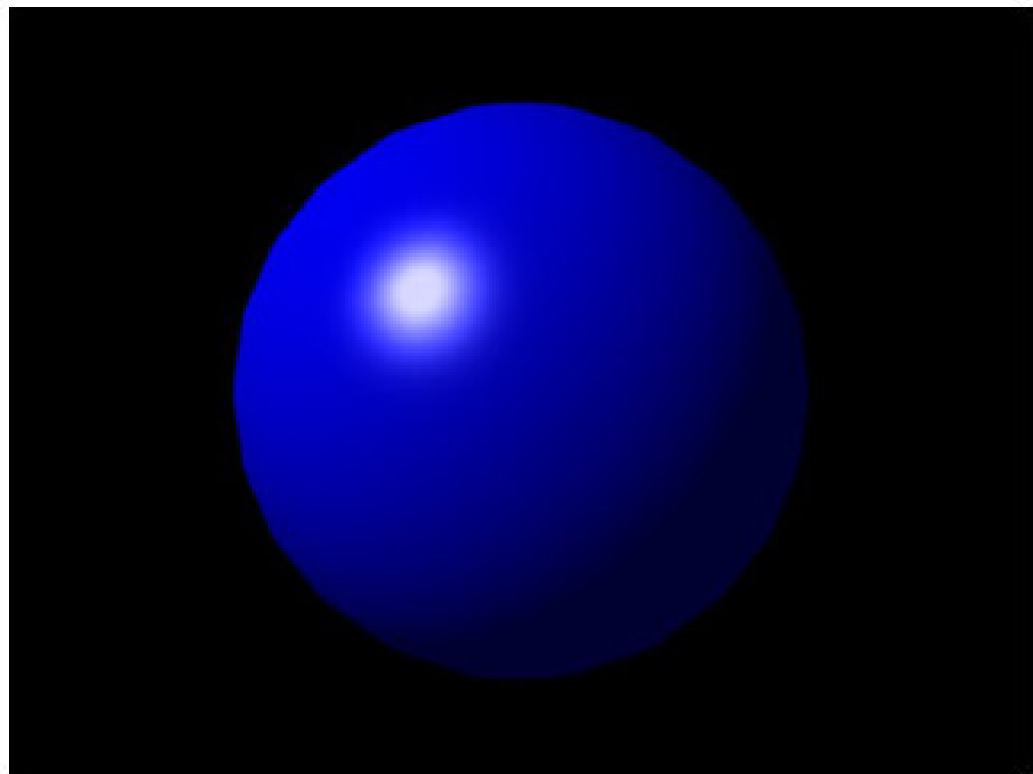
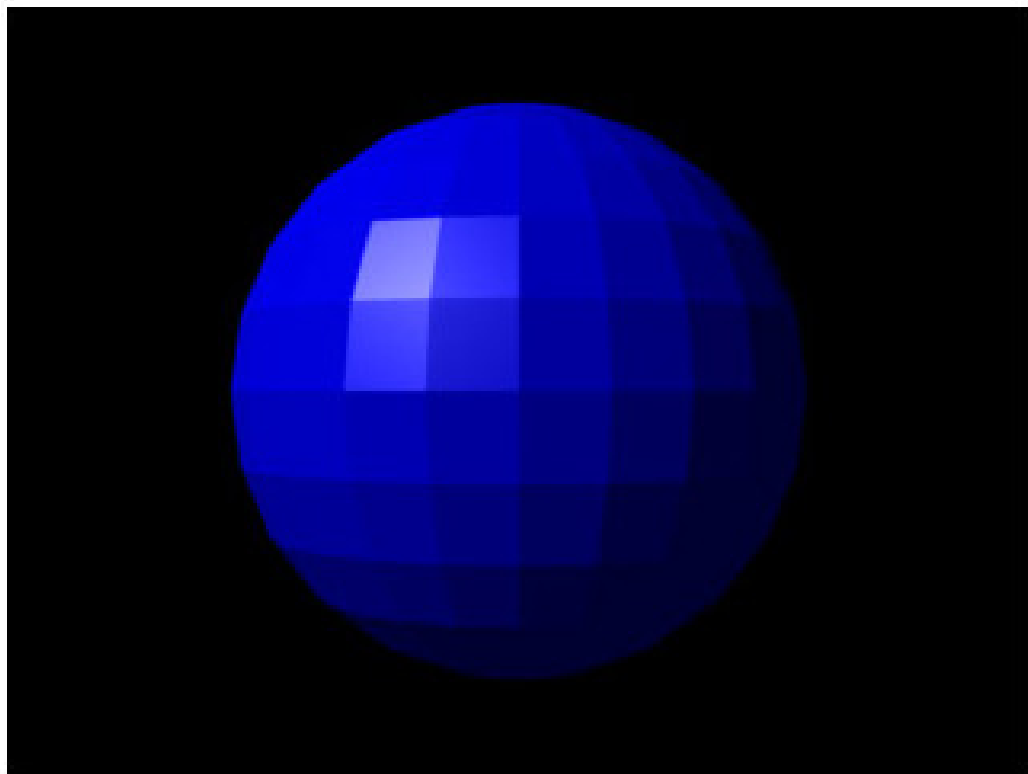
サンプルプログラム

- 色を塗る
- 色の点滅
- 位置情報によるグラデーション
- 波を動かす
- 同心円の波 1
- 同心円の波 2
- シェーダーを平面にハイトマッピング
- シェーダーを球体にハイトマッピング

Shaderとは何か？

Shaderとは

- もともとは、3DCGにおいて、シェーディング（陰影処理）を行うコンピュータプログラムのこと



例: Phong Shading

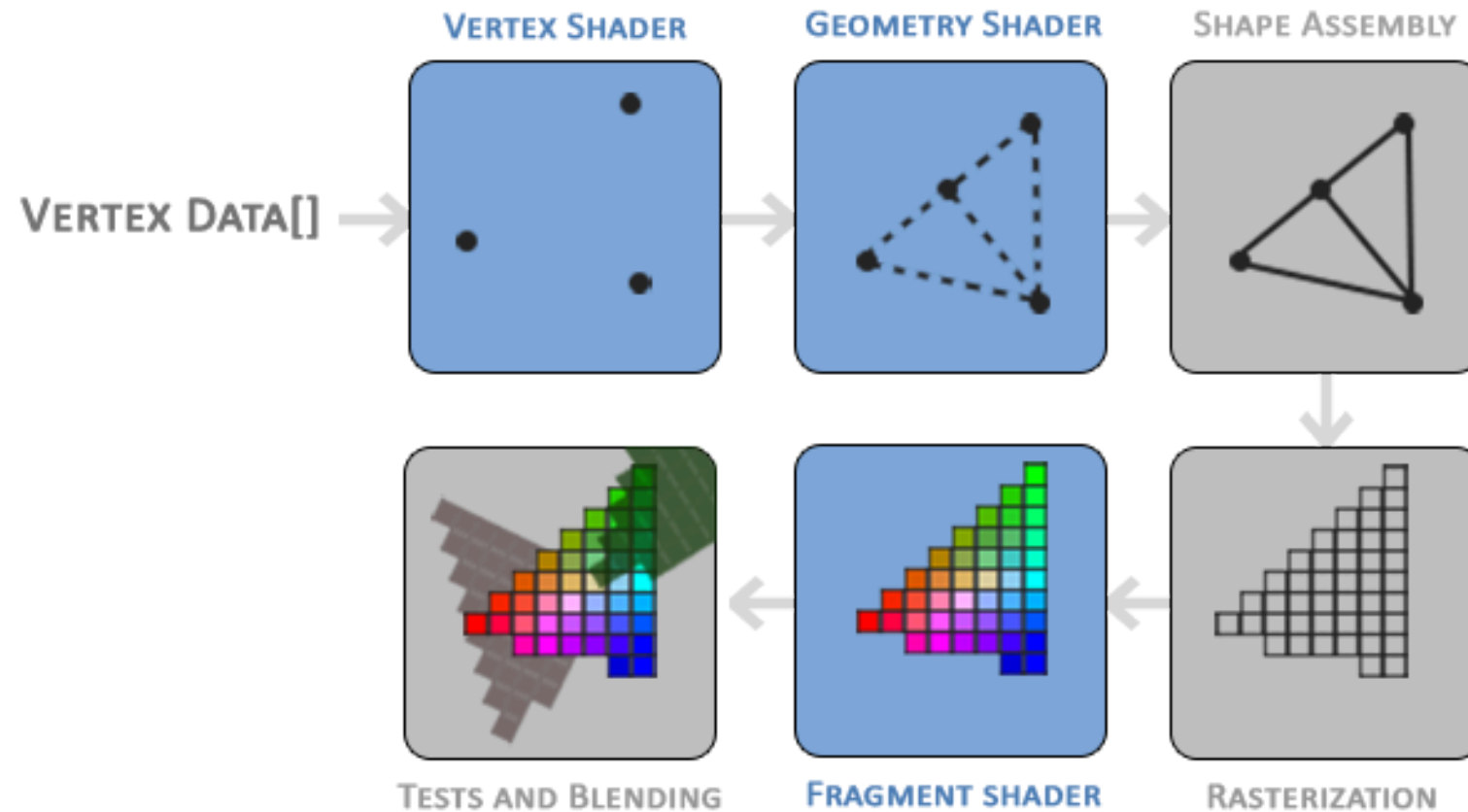
- 従来は、開発者やデザイナーは、グラフィクスカード (GPU) に固定機能として実装された定形の処理しか使えなかった (固定機能シェーダー)
- 2000年代に入って：プログラマブル・シェーダーの登場
- ブラックボックスだったシェーダー自体が、プログラム可能になった
- 3D描画ライブラリによってシェーダー言語は異なる
 - OpenGL → GLSL
 - Direct 3D → HLSL
- TochDesignerでは、描画にOpenGLを使用
 - GLSLをShader言語として用いる
 - GLSL TOPでShaderを表示可能

Shaderの種類

- 頂点シェーダー (Vertex Shader)
 - 入力された頂点を座標変換するための機能
- ジオメトリシェーダー (Geometry Shader)
 - オブジェクト内の頂点の集合を加工して新しいプリミティブ図形を生成
- フラグメントシェーダー (Fragment Shader)
 - ピクセルを操作する。画面上の膨大なピクセル情報を、高い並列処理性能を持つGPUで実行することにより、CPUで実行するよりもはるかに高いパフォーマンスを実現
 - ピクセルシェーダー (Pixel Shader) と呼ばれることも

→ 今回は主にフラグメントシェーダーを扱います!

頂点のデータの集合から画面に画像が表示されるまでの流れ



Shaderの教材

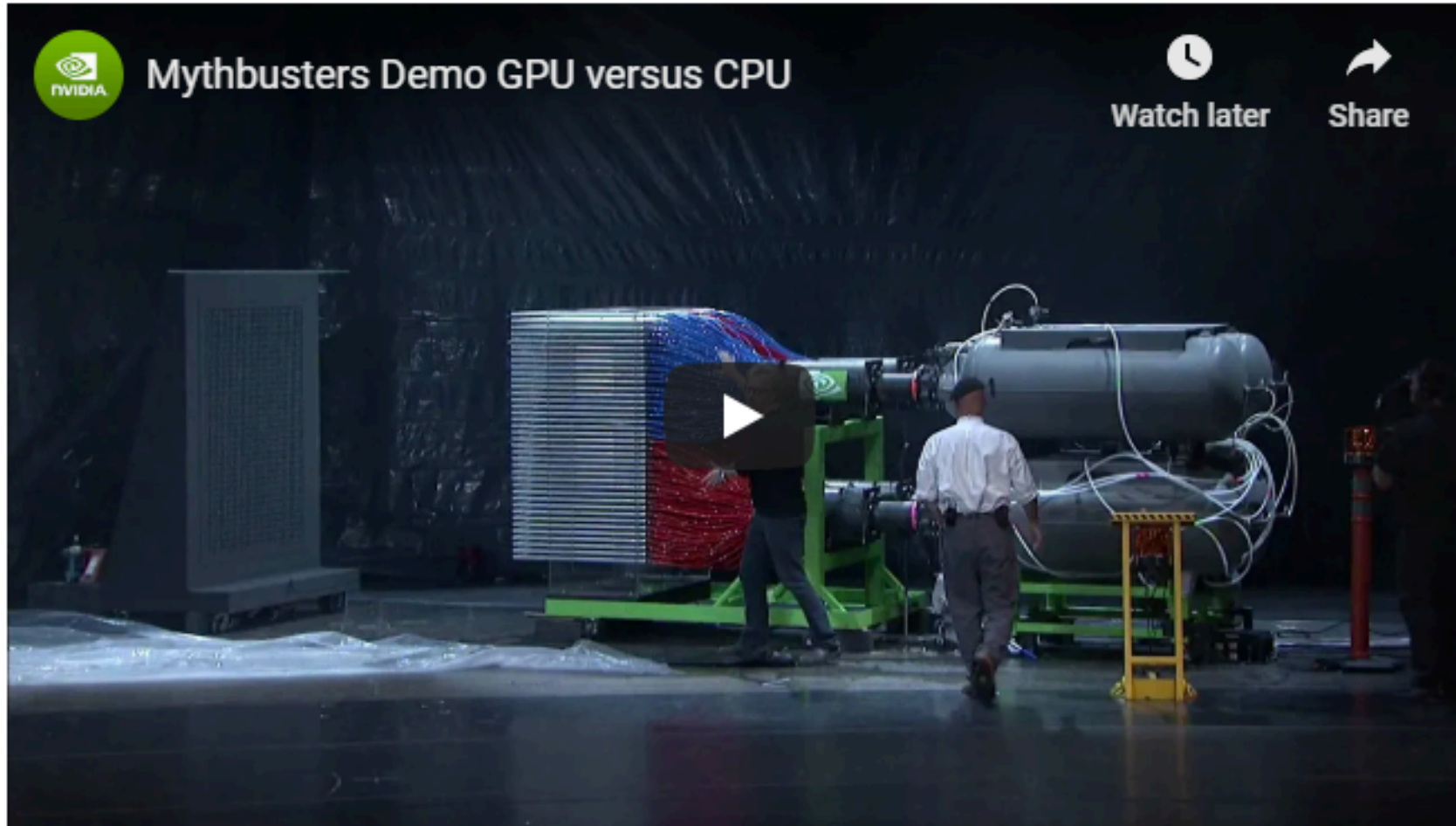
現在、すばらしいオンライン教材の執筆が進行中!



The Book of Shaders

- The Book of Shaders - Patricio Gonzalez Vivo
- <http://patriciogonzalezvivo.com/2015/thebookofshaders/>

Mythbusters Demo GPU versus CPU



<https://youtu.be/ZrJeYFxpUyQ?si=3n-9e01FzwNxxkwsR>

Shaderのバージョン

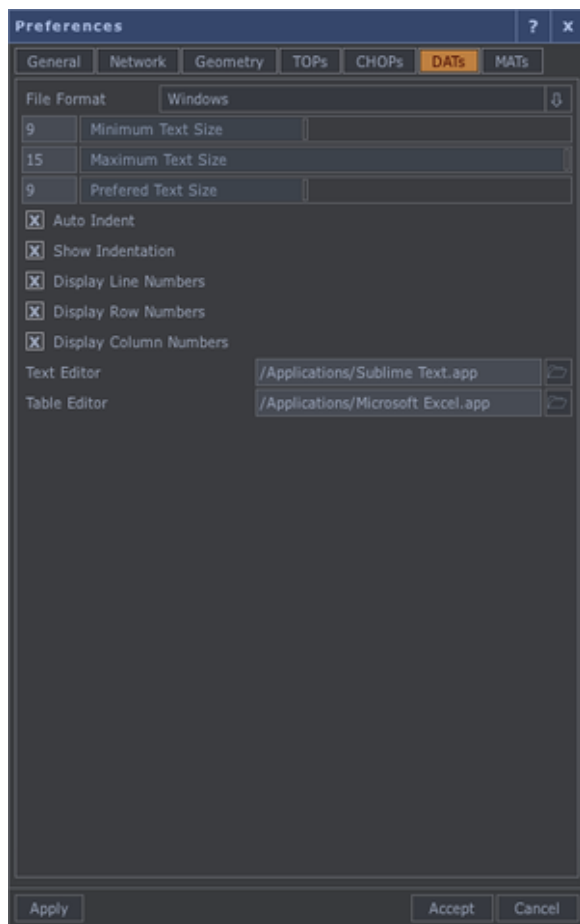
- GLSLには様々なバージョンがあって、ちょっとやっかい
- TouchDesignerでは、GLSL 4.60 (OpenGL 4.6) に対応している
- 2024年現在のGLSL (及びOpenGLの最新版)

OpenGLバージョン	GLSLバージョン	#versionディレクティブ
1.5	1.0	なし
2.0	1.1	#version 110
2.1	1.2	#version 120
3.0	1.3	#version 130
3.1	1.4	#version 140
3.2	1.5	#version 150
3.3	3.3	#version 330

OpenGLバージョン	GLSLバージョン	#versionディレクティブ
4.0	4.0	#version 400
4.1	4.1	#version 410
4.2	4.2	#version 420
4.3	4.3	#version 430
4.4	4.4	#version 440
4.5	4.5	#version 450
4.6	4.6	#version 460

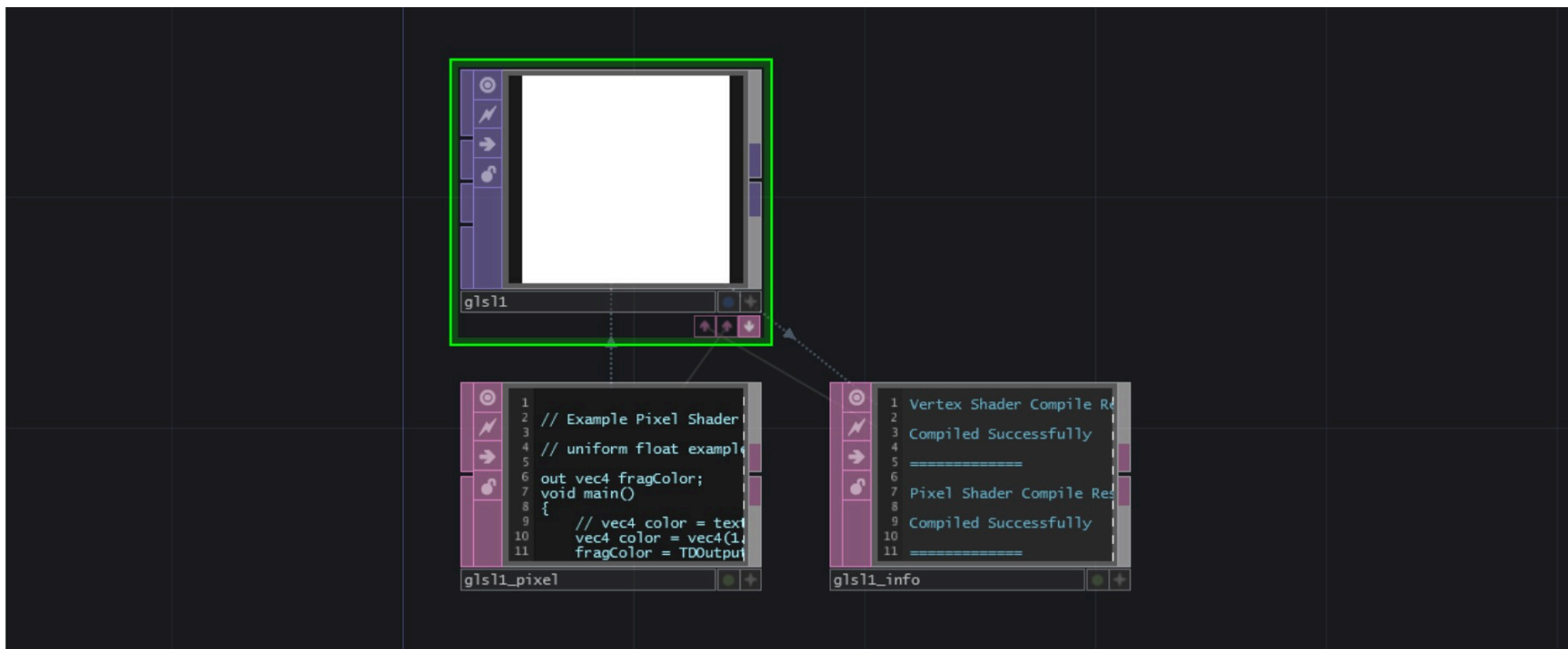
TouchDesingnerでGLSLを使う

- テキストエディタの設定 : Preferences > DATs でText Editorの場所を設定
- 普段使用しているエディター (VScodeなど) を設定しておく



画面にGLSL TOPを配置

- まずは、画面上にGLSL TOPを配置
- 自動的に、glsl_pixelとglsl_infoの2つのDATがセットで配置される
- glsl1_pixel DAT のパラメータから「Edit」ボタンを押す
- 設定したテキストエディターでGLSLを編集可能になる



- 初期状態で以下のフラグメントシェーダーが記述されている
- このファイルを編集していく

```
// Example Pixel Shader
// uniform float exampleUniform;

out vec4 fragColor;

void main() {
    // vec4 color = texture(sTD2DInputs[0], vUV.st);
    vec4 color = vec4(1.0);
    fragColor = TDOutputSwizzle(color);
}
```

- このコードはだいたい以下の意味

```
// ピクセルシェーダーのサンプル
// uniform float exampleUniform; ← 外部からの入力例
out vec4 fragColor; // 最終出力する場所

void main() //メイン関数
{
    // ↓ 外部テクスチャーの色の参照方法
    // vec4 color = texture(sTD2DInputs[0], vUV.st);

    vec4 color = vec4(1.0); //色を指定(白)

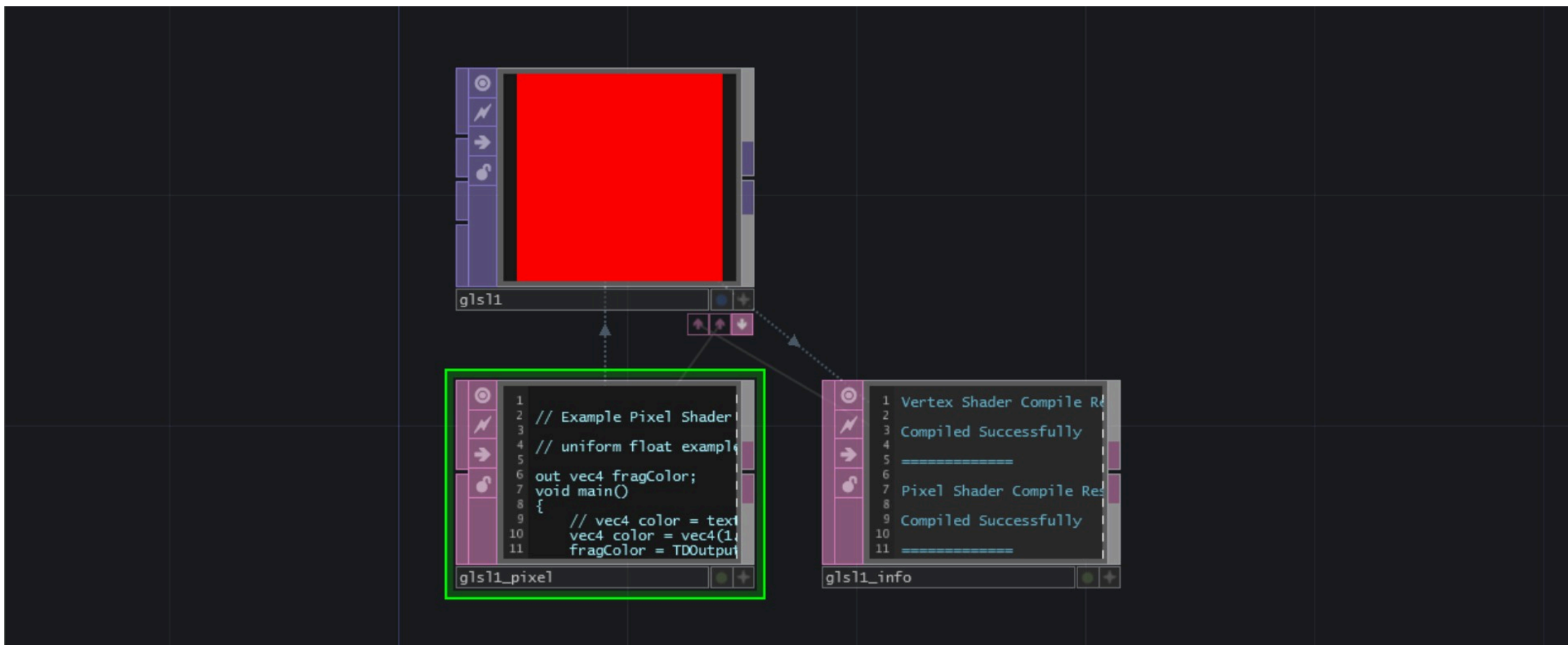
    //最終出力へ、TDOutputSwizzle()はMacとWinのずれを解消する関数
    fragColor = TDOutputSwizzle(color);
}
```

- 試しに、vec4 colorの値を変化させてみる 例 vec4(1.0, 0.0, 0.0, 1.0)
- 何が変わるか？

```
out vec4 fragColor;

void main() {
    // vec4で色を指定 (R:1.0, G:0.0, B:0.0, A:1.0)
    vec4 color = vec4(1.0, 0.0, 0.0, 1.0);
    fragColor = TDOutputSwizzle(color);
}
```

- 赤い色に変化した!
- `color = vec4(Red, Green, Blue Alpha);`



プログラムを読み解いてみる

- GLSLのプログラムは、まず始めに main関数が実行される
- 最終出力をfragColorと宣言 `out vec4 fragColor;`
- main関数で演算した最終的なピクセルの色の値は、fragColorに出力する
- 変数や関数の書き方は、ほぼ、C / C++ の書き方を踏襲している
- int, float, bool などはC / C++と同じように使用可能
- vec2, vec3, vec4 など、GLSL独自の型も存在している

代表的なGLSLの型

- vec2 : float型による2次元のベクトル
- vec3 : float型による3次元のベクトル
- vec4 : float型による4次元のベクトル
- ivec2 : int型による2次元のベクトル
- ivec3 : int型による3次元のベクトル
- ivec4 : int型による4次元のベクトル
- mat2 : 2x2要素を持つfloat型の行列
- mat3 : 3x3要素を持つfloat型の行列
- mat4 : 4x4要素を持つfloat型の行列
- Sampler1D : 1次元のテクスチャ
- Sampler2D : 2次元のテクスチャ
- Sampler3D : 3次元のテクスチャ

Uniforms - TouchDesignerからShaderへ値を送出

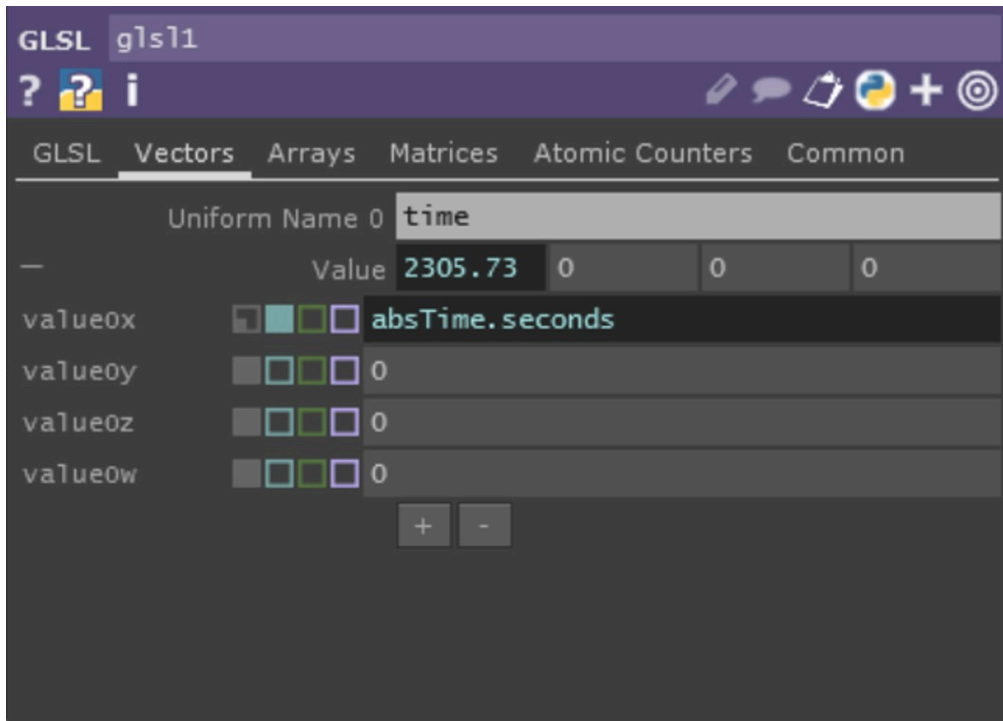
TouchDesignerと、GLSLの情報のやりとりのイメージ



経過時間を送る - 色の点滅

- 送出したUniformsをShaderで活用してみる!
 - まず始めに経過時間を送出
 - TouchDesigner側 : `absTime.seconds`
 - GLSL側 : `float`

- GLSL TopのパラメータのVectorsを開く
- 以下のパラメータを設定して、経過時間をGLSLに送出する
 - Uniform Name 0 : time
 - value0x : absTime.seconds



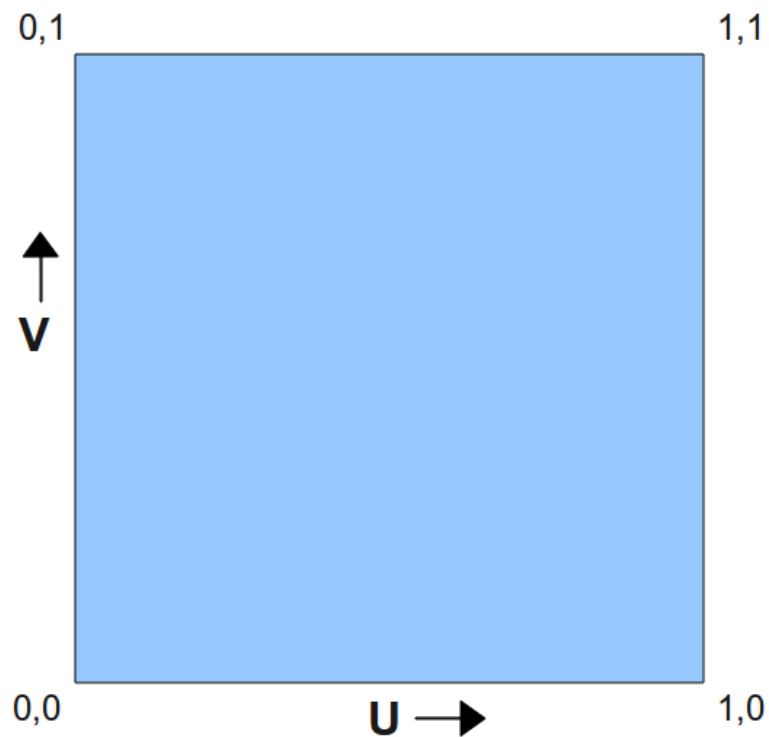
- 時間と三角関数と絶対値で色を点滅させる
- 色が激しく点滅する (はず) !

```
uniform float time;  
out vec4 fragColor;  
  
void main() {  
    float r = abs(sin(time * 10.0)); // 赤の点滅  
    float g = abs(sin(time * 12.0)); // 緑の点滅  
    float b = abs(sin(time * 14.0)); // 青の点滅  
    vec4 color = vec4(r, g, b, 1.0); // vec4(RGBA)で色を指定  
    fragColor = TDOutputSwizzle(color);  
}
```

経過時間を送る - 色の点滅

ピクセルシェーダーの座標系

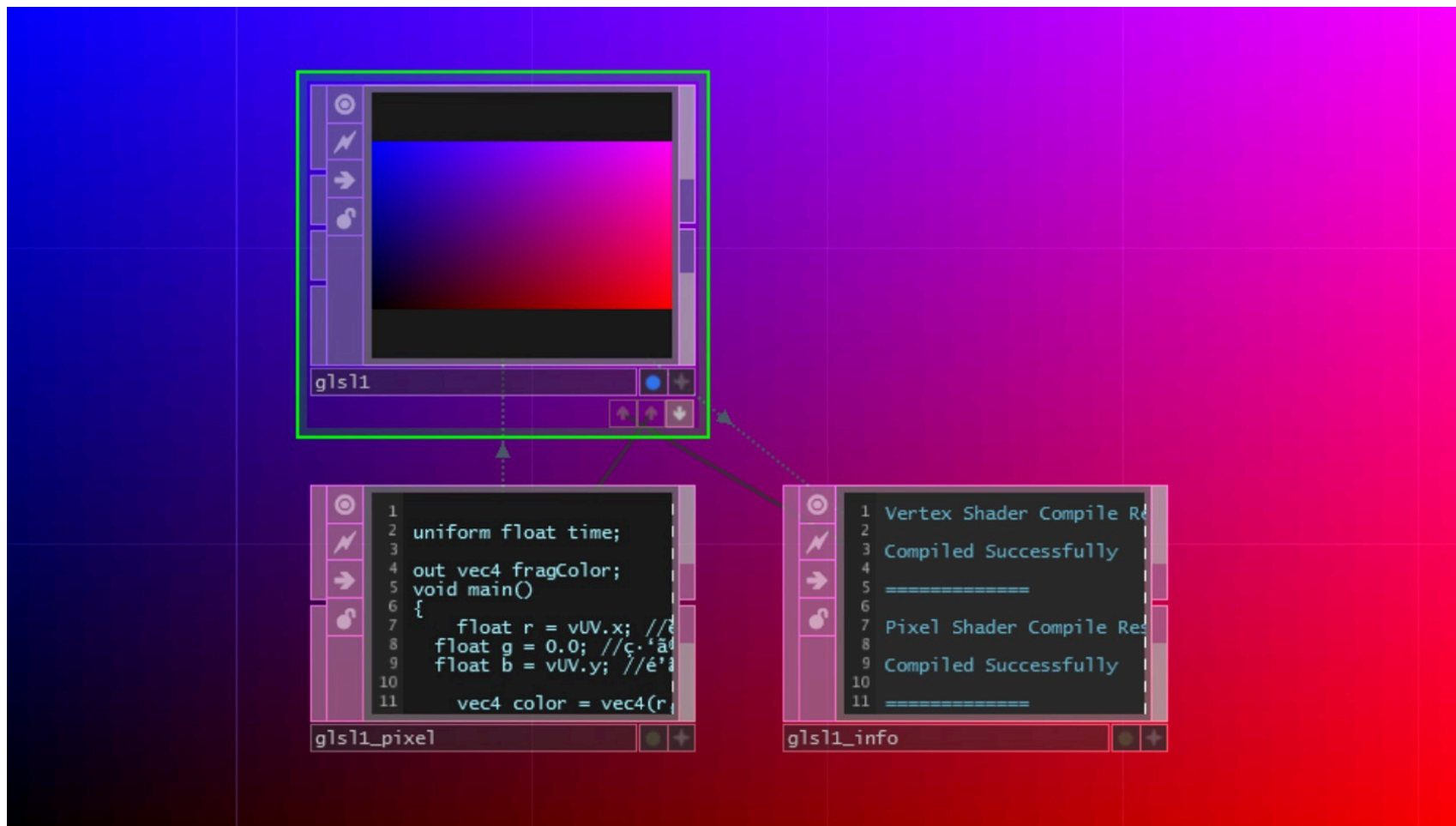
- ピクセルシェーダーのコード - テクスチャ上のピクセルの1つ1つにプログラムが埋め込まれているイメージ
- 自分自身のピクセルの場所を知るには? → $vUV.x$ と $vUV.y$ を参照
- 左下が(0.0, 0.0) 右上が (1.0, 1.0)



試しにグラデーションを描いてみる!

```
uniform float time;  
out vec4 fragColor;  
  
void main() {  
    float r = vUV.x;    //x方向: 赤のグラデーション  
    float g = 0.0;      //緑は0.0  
    float b = vUV.y;    //y方向: 青のグラデーション  
    vec4 color = vec4(r, g, b, 1.0);  
    fragColor = TDOutputSwizzle(color);  
}
```

なめらかなグラデーションが描けた!



移動する波 (座標情報 + 経過時間)

- vUV.xy による座標の情報と経過時間 (time) を組合せてみる
- sin関数で座標の情報と経過時間を使用して移動する波を作ってみる

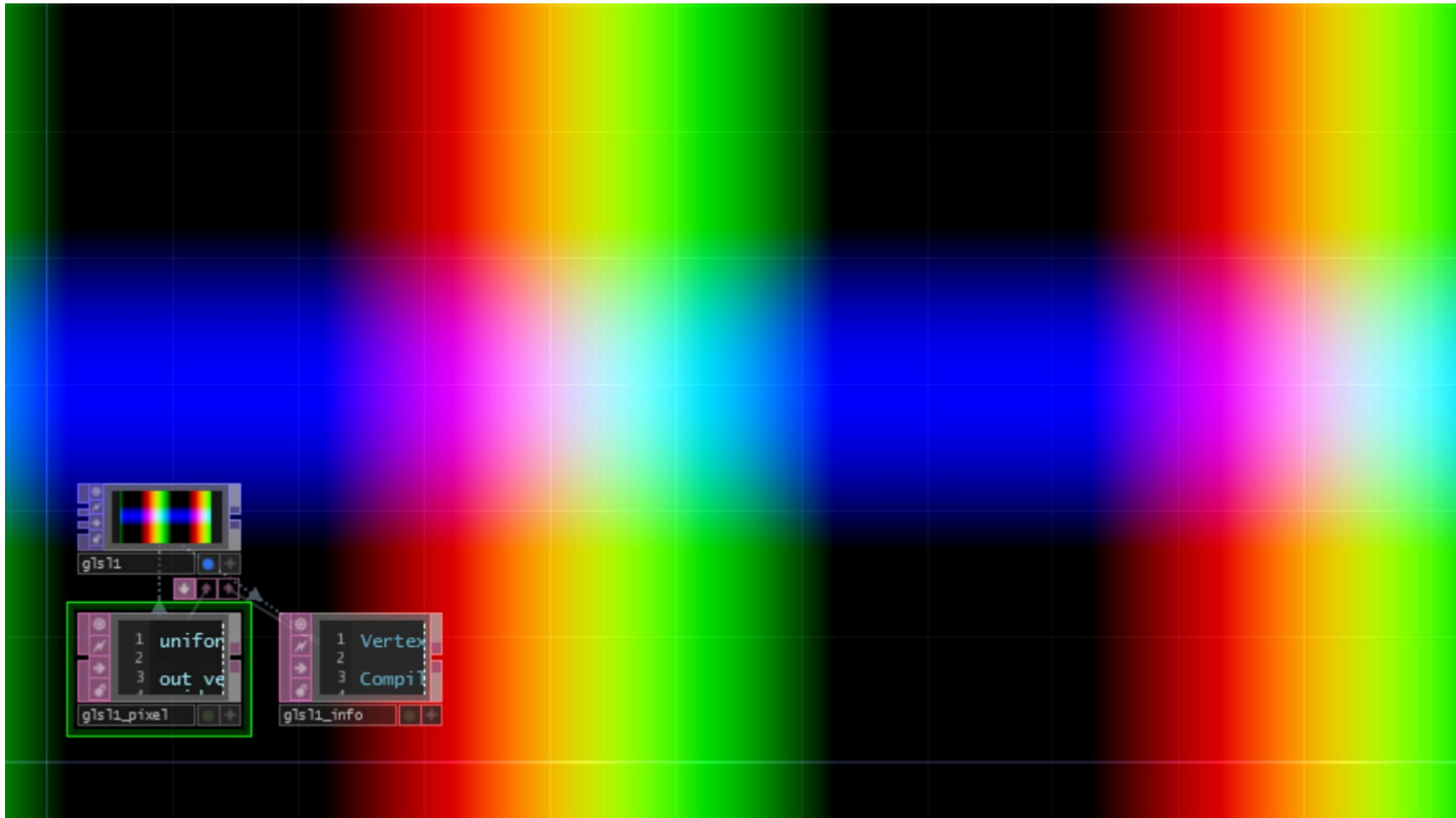
移動する波

```
uniform float time;
out vec4 fragColor;

void main() {
    float r = sin(time * 10.0 + vUV.x * 12.0); // 赤
    float g = sin(time * -10.0 + vUV.x * 12.0); // 緑
    float b = sin(time * 10.0 + vUV.y * 8.0);   // 青
    float a = 1.0; // 透明度

    vec4 color = vec4(r, g, b, a);
    fragColor = TDOutputSwizzle(color);
}
```

RGBのストライプが移動する!



拡がる同心円

- もう少し複雑な図形を描いてみる
- 同心円を描く
- ポイント：中心点からの距離を計測してその値で形を描く
- GLSLで距離を測る関数 → `length()`

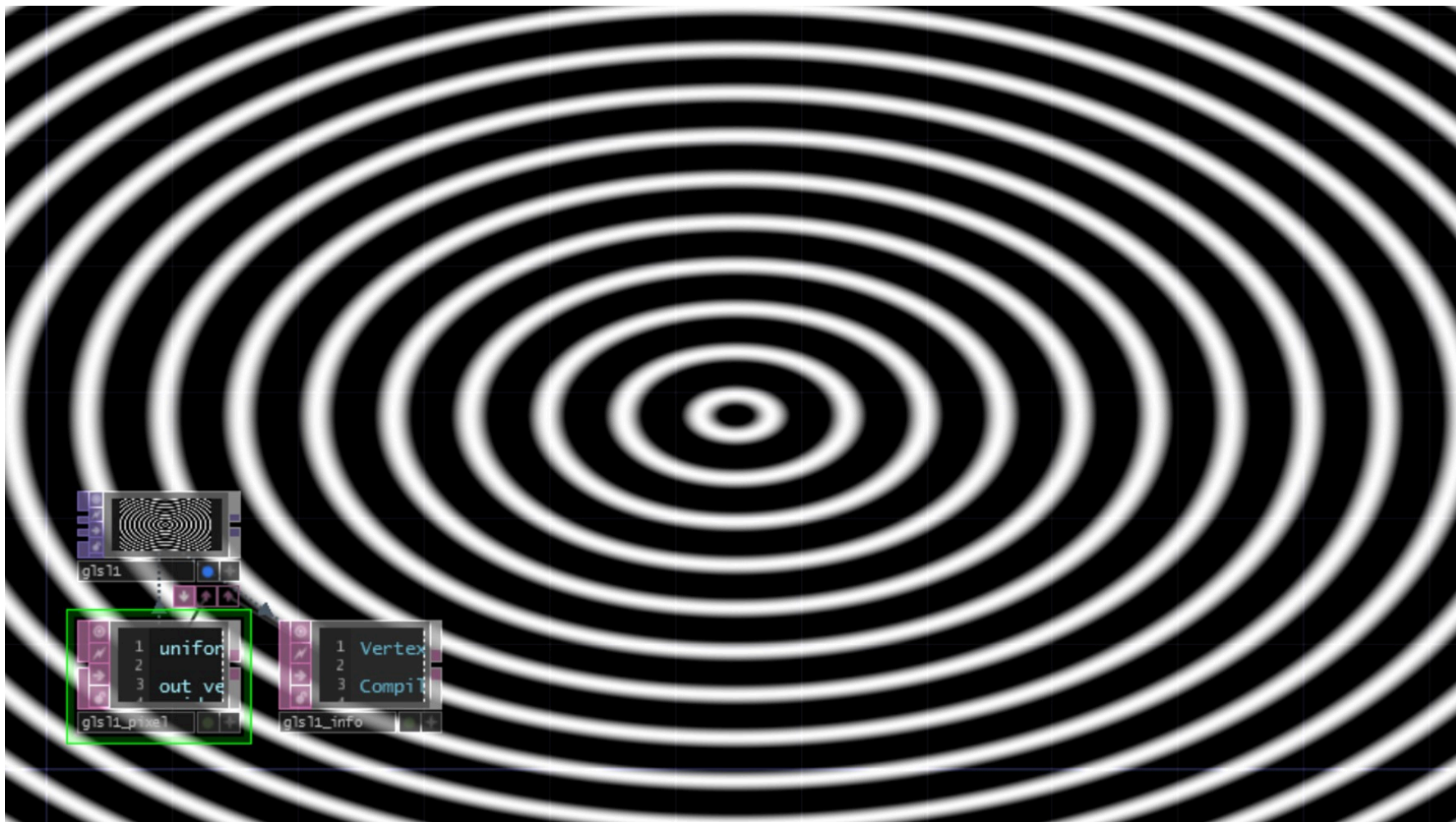
```
float len = length(startPos, endPos);
```

拡がる同心円

```
uniform float time;
out vec4 fragColor;

void main() {
    //画面の中心からの距離を算出
    float len = length(vec2(0.5, 0.5) - vUV.xy);
    //画面中心からの距離でsin波を生成し同心円状の波に
    float br = sin(len * 120 - time * 40.0);
    //ピクセルの色に設定
    vec4 color = vec4(br, br, br, 1.0);
    fragColor = TDOutputSwizzle(color);
}
```

中心から広がる同心円が描けた!



拡がる同心円 (真円バージョン)

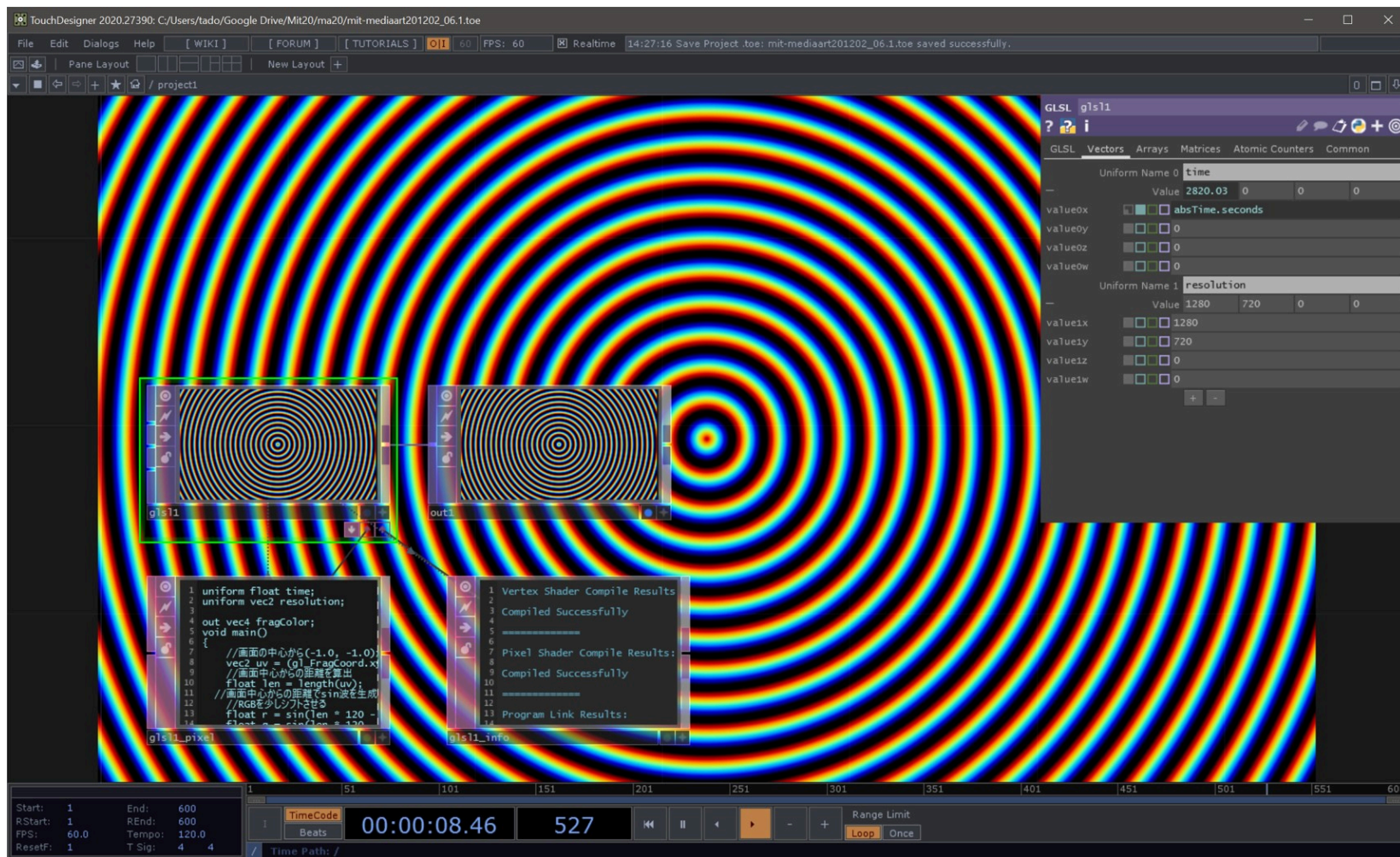
- 円の幅と高さを揃えて楕円ではなく真円にしてみる
- v_{UV} を画面の解像度の幅で x と y 座標の双方を割り算する
- さらに、RGBで拡がる速度を変化させてみる

拡がる同心円 (真円バージョン)

```
uniform float time;
uniform vec2 resolution;
out vec4 fragColor;

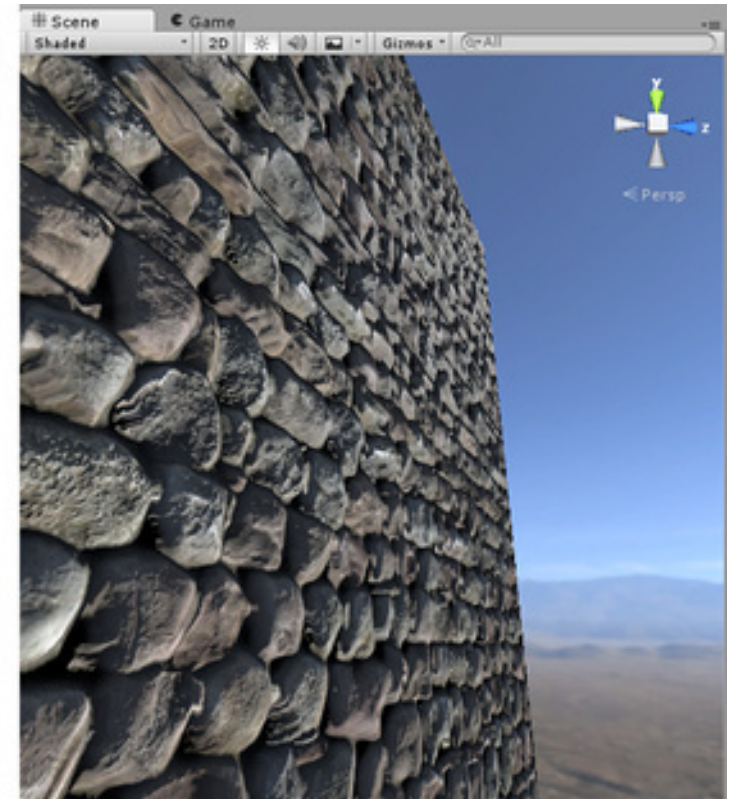
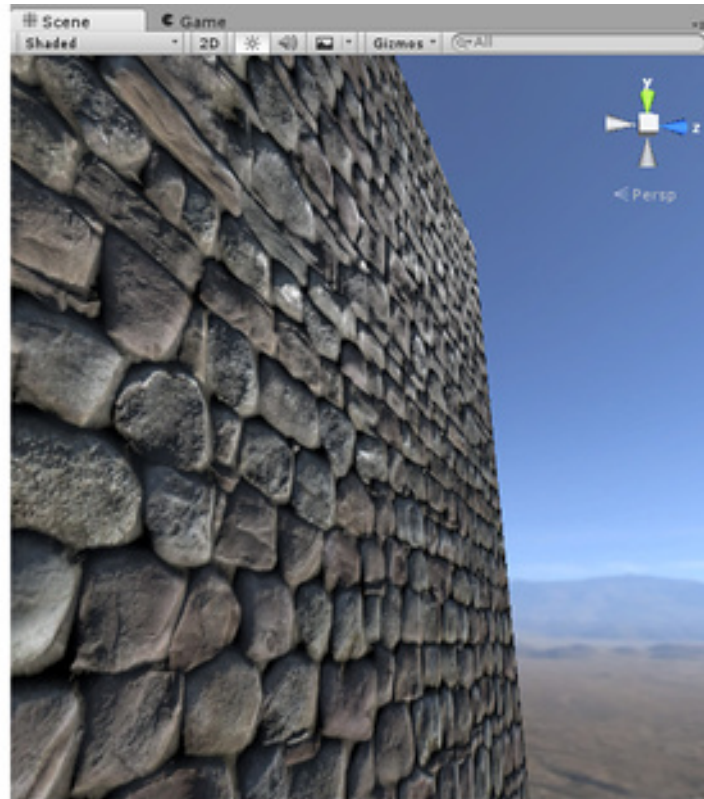
void main() {
    //画面の中心から(-1.0, -1.0)から(1.0, 1.0)の範囲の座標に変換し縦横の比率を揃える
    vec2 uv = (gl_FragCoord.xy * 2.0 - resolution.xy) / vec2(resolution.x, resolution.x);
    //画面中心からの距離を算出
    float len = length(uv);
    //画面中心からの距離でsin波を生成し同心円状の波に
    //RGBを少しシフトさせる
    float r = sin(len * 120 - time * 40.0);
    float g = sin(len * 120 - time * 40.0 - 1.0);
    float b = sin(len * 120 - time * 40.0 - 2.0);
    vec4 color = vec4(r, g, b, 1.0);
    fragColor = TDOutputSwizzle(color);
}
```


円が真円になった!

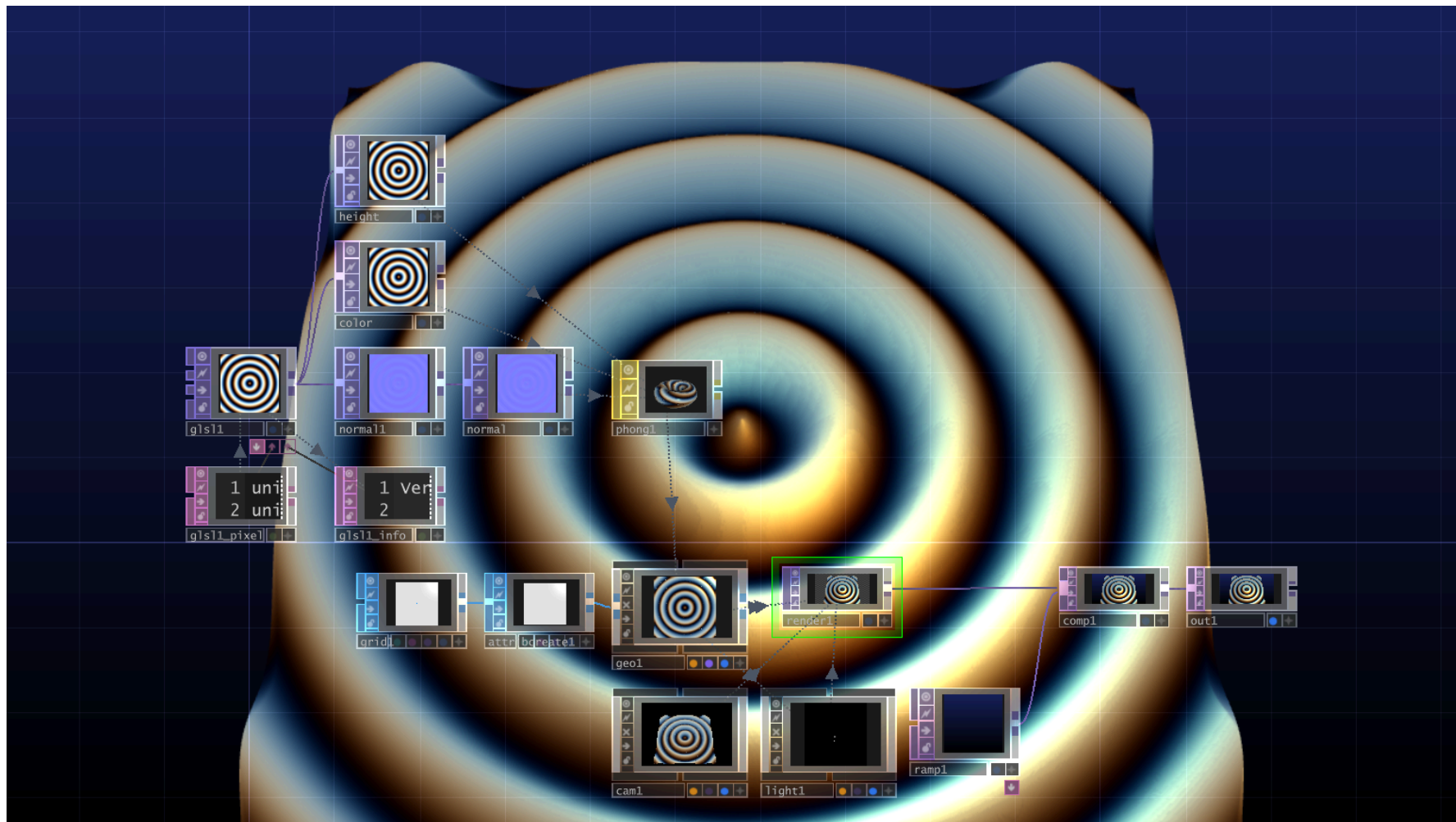


GLSL応用: 生成されたイメージを3D化

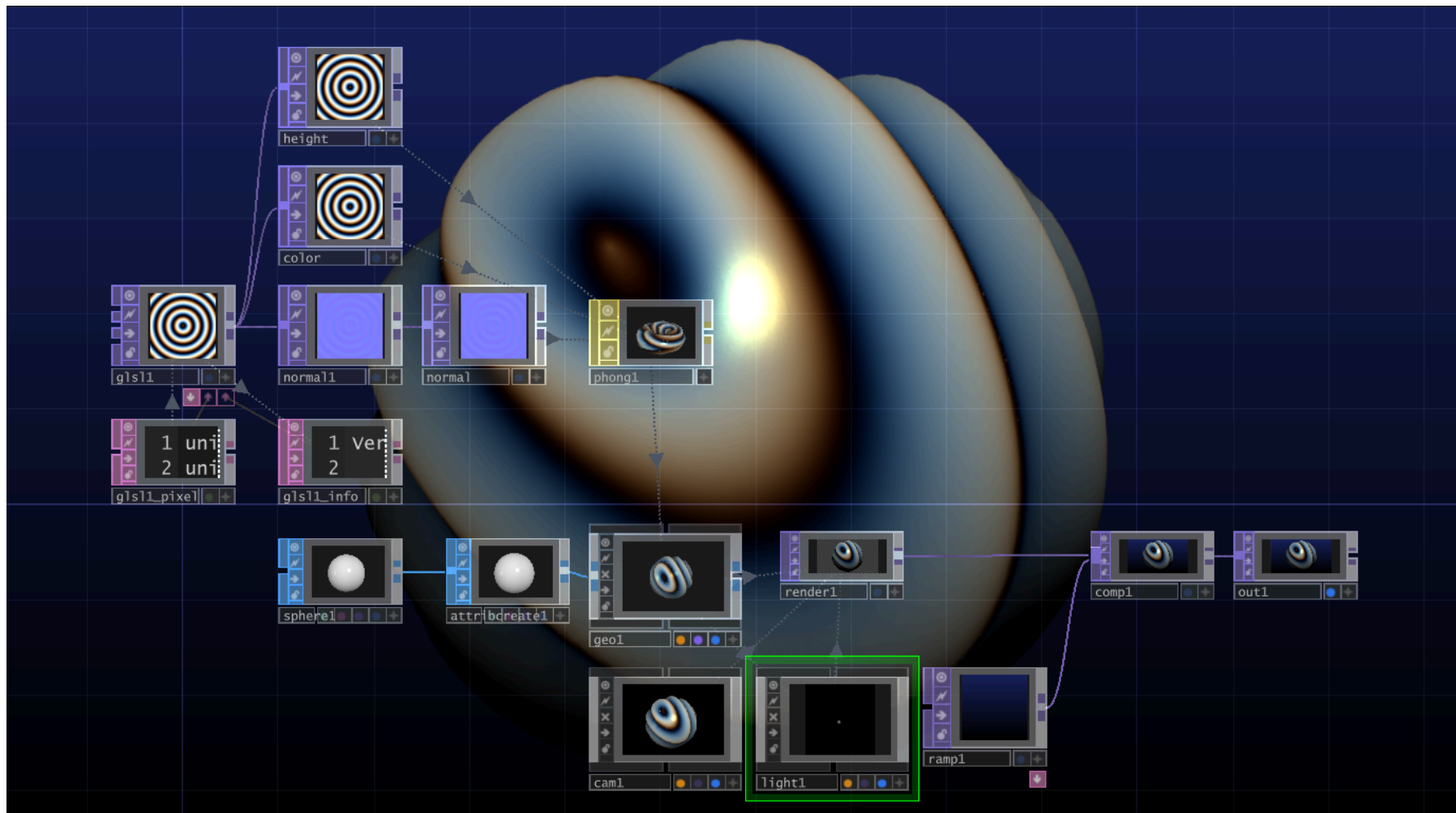
- 同心円のアニメーションのパターンを用いて凸凹を生成
- ハイトマップ (Height Map) という手法を用いている
- ゲームエンジンなどで使用されている手法



GLSLのパターンを立体化 (詳細は実際のプログラムで解説)



さらに球体にも適用してみる!



アンケート

本日の演習に参加した方は以下のアンケートに回答してください。

- [アンケート](#)